LEAF AND NON-LEAF NODE COUNTING IN A BINARY TREE

Problem Statement:

Develop a C++ program to count and display the number of leaf nodes and non-leaf nodes in a binary tree. The program should build the binary tree by inserting nodes, check each node for the presence of left and right children, and classify it as a leaf node (no children) or a non-leaf node (at least one child). It should then display the leaf count, the non-leaf count and the total number of nodes, correctly handling an empty tree and a tree with a single node.

Objective:

The objective of this project is to implement a binary tree in C++ and efficiently count the leaf nodes and non-leaf nodes present in it. It aims to demonstrate how each node is checked for the presence of left and right children to enable accurate counting using traversal. The project also strengthens understanding of tree creation, insertion, traversal, and recursion, while handling edge cases where the tree is empty or contains only a single node.

Algorithm:

Step 1: Start the program and set root = NULL.

Step 2: Define a node with data, left and right pointers. A new node has both pointers NULL.

Step 3: Read a key to insert. If the tree is empty, make the new node the root.

Step 4: Otherwise, start at the root and move left if key < data, or right if key > data. Stop when the next link is NULL.

Step 5: If the key is smaller than the parent, attach the new node as parent->left; if larger, attach it as parent->right. Ignore duplicate keys.

Step 6: To count the nodes, call the counting function with the root and set leaf = 0 and nonleaf = 0.

Step 7: If the current node is NULL (empty tree or end of a branch), return without counting.

Step 8: If node->left = NULL and node->right = NULL, the node is a leaf node. Increase leaf by 1.

Step 9: Otherwise, the node has at least one child, so it is a non-leaf node. Increase nonleaf by 1.

Step 10: Repeat steps 7 to 9 recursively for the left subtree and then for the right subtree.

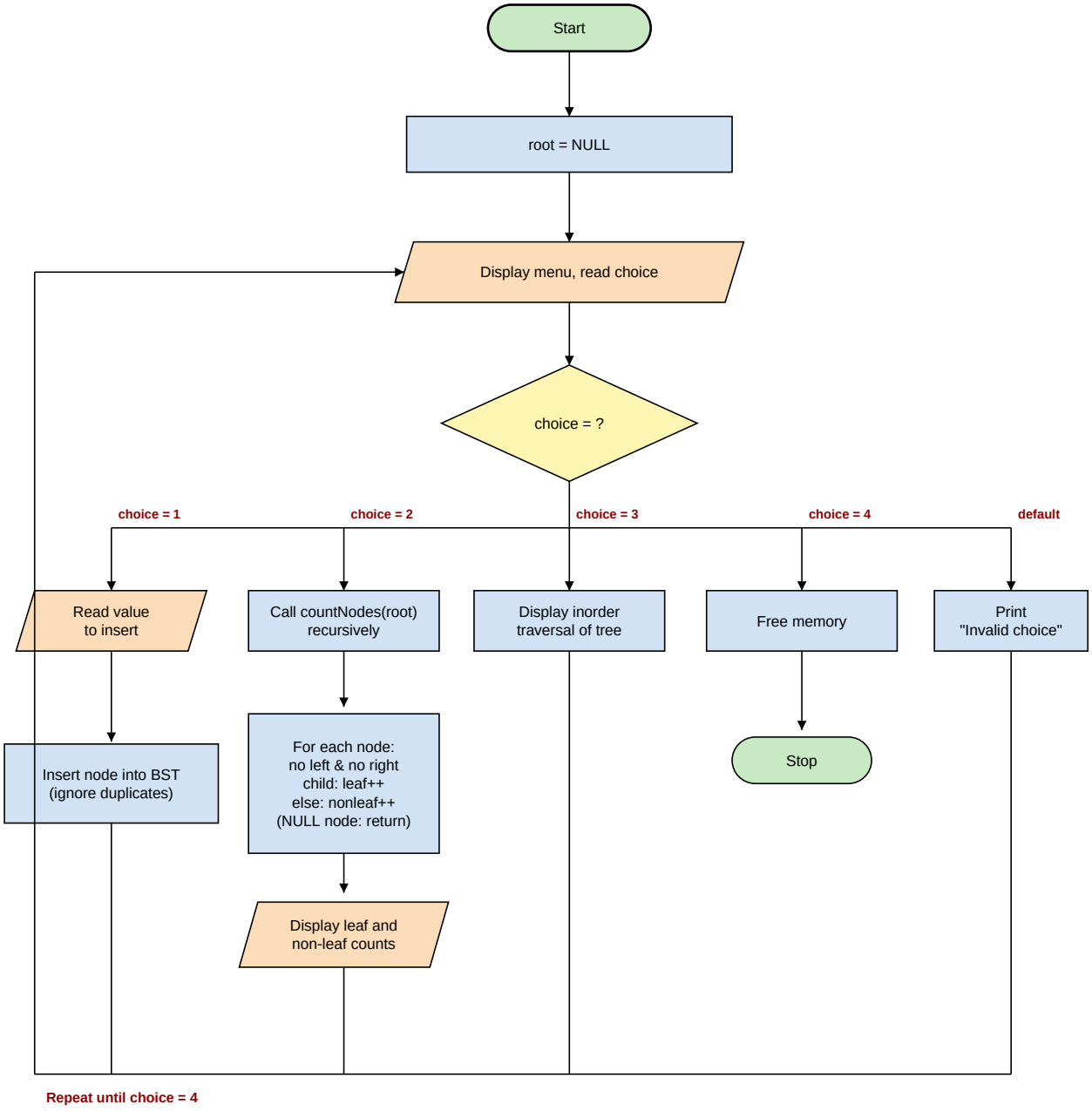
Step 11: If the tree has only one node (the root), it is counted as a leaf and nonleaf remains 0.

Step 12: Display the leaf count, the non-leaf count and the total number of nodes (leaf + nonleaf).

Step 13: Free the allocated memory and stop the program.

Flowchart:

Flowchart: Leaf and Non-Leaf Node Counting in a Binary Tree



Program: leaf_nonleaf_count.cpp (menu-driven, binary tree)